6

ASCII ART

In the 1990s, when email ruled and graphics capabilities were limited, it was common to include a signature in your email that contained a graphic made of text, commonly called *ASCII art*. (ASCII is simply a character-encoding scheme.) **Figure 6-1** shows a couple of examples. Although the internet has made sharing images immeasurably easier, the humble text graphic isn't quite dead yet.

ASCII art has its origins in typewriter art created in the late 1800s. In the 1960s, when computers had minimal graphics processing hardware, ASCII was used to represent images. These days, ASCII art continues as a form of expression on the internet, and you can find a variety of creative examples online.

Figure 6-1: *Examples of ASCII art*

In this project, you'll use Python to create a program that generates ASCII art from graphical images. The program will let you specify the width of the output (the number of columns of text) and set a vertical scale factor. It also supports two mappings of grayscale values to ASCII characters: a sparse 10-level mapping and a more finely calibrated 70-level mapping.

To generate your ASCII art from an image, you'll learn how to do the following:

- Convert images to grayscale using `Pillow`, a fork of the Python Imaging Library (PIL).
- Compute the average brightness of a grayscale image using `numpy`.
- Use a string as a quick lookup table for grayscale values.

How It Works

This project takes advantage of the fact that from a distance, we perceive grayscale images as the average value of their brightness. For example, in **Figure 6-2**, you can see a grayscale image of a building and, next to it, an image filled with the average brightness value of the building image. If you look at the images from across the room, they will look similar.

ASCII art is generated by splitting an image into tiles and replacing each tile with an ASCII character, based on the tile's average brightness value. Brighter tiles are replaced with sparser ASCII characters (that is, characters that contain a lot of whitespace), such as a period (`.`) or colon (`:`), while darker tiles are replaced with denser ASCII characters, such as an ampersand (`@`) or dollar sign (`$`). From a distance, since our eyes have limited resolution, we sort of see the "average" values in ASCII art while losing the details that would otherwise make the art look less real.

Figure 6-2: *The average value of a grayscale image*

This program will take a given image and first convert it to 8-bit grayscale so that each pixel has a grayscale value in the range [0, 255] (the range of an 8-bit integer). Think of this 8-bit value as the pixel's *brightness*, with 0 being black, 255 being white, and the values in between being shades of gray.

Next, it will split the image into a grid of $M \times N$ tiles (where M is the number of rows and N the number of columns in the ASCII image). The program will then calculate the average brightness value for each tile in the grid and match it with an appropriate ASCII character by predefining a *ramp* (an increasing set of values) of ASCII characters to

represent grayscale values in the range [0, 255]. It will use these ramp values as a lookup table for the brightness values.

The finished ASCII art is just a bunch of lines of text. To display the text, you'll use a constant-width (also called *monospace*) font such as Courier because unless each text character has the same width, the characters in the image won't line up properly along a grid, and you'll end up with unevenly spaced and scrambled output.

The *aspect ratio* (the ratio of width to height) of the font used also affects the final image. If the aspect ratio of the space taken up by a character is different from the aspect ratio of the image tile the character is replacing, the final ASCII image will appear distorted. In effect, you're trying to replace an image tile with an ASCII character, so their shapes need to match. For example, if you were to split your image into square tiles and then replace each of the tiles with a font where characters are taller than they are wide, the final output would appear stretched vertically. To address this issue, you'll scale the rows in your grid to match the Courier aspect ratio. (You can send the program command line arguments to modify the scaling to match other fonts.)

In sum, here are the steps the program takes to generate the ASCII image:

1. Convert the input image to grayscale.
2. Split the image into $M \times N$ tiles.
3. Correct M (the number of rows) to match the image and font aspect ratio.
4. Compute the average brightness for each image tile and then look up a suitable ASCII character for each.
5. Assemble rows of ASCII character strings and print them to a file to form the final image.

Requirements

In this project, you'll use `Pillow`, the friendly fork of the Python Imaging Library, to read in the images, access their underlying data, and create and modify them. You'll also use the `numpy` library to compute averages.

The Code

You'll begin by defining the grayscale levels used to generate the ASCII art. Then you'll look at how the image is split into tiles and how average brightness is computed for those tiles. Next, you'll work on replacing the tiles with ASCII characters to generate the final output. Finally, you'll set up command line parsing for the program to allow users to specify the output size, output filename, and so on.

For the full project code, skip to **"The Complete Code"** on **page 109**.
You can also download the code for this project from ***https://github.com/mkvenkit/pp2e/blob/main/ascii/ascii.py***.

Defining the Grayscale Levels and Grid

As the first step in creating your program, define the scales you'll use to convert image brightness values to ASCII characters as global values.

```
# 70 levels of gray

gscale1 = "$@B%8&WM#*oahkbdpqwmZO0QLCJUYXzcvunxrjft/\|()1{}
[]?-_+~<>i!lI;:,\"^`'. "

# 10 levels of gray

gscale2 = "@%#*+=-:. "
```

The value `gscale1` is a 70-level grayscale ramp, while `gscale2` is a simpler 10-level grayscale ramp. Both of these values are stored as

strings, with a range of characters that progress from darkest to lightest. The program will use the `gscale2` ramp by default, but you'll include a command line option to use the more nuanced `gscale1` ramp instead.

NOTE *To learn more about how characters are represented as grayscale values, see Paul Bourke's "Character Representation of Grey Scale Images" at* **http://paulbourke.net/dataformats/asciiart/**.

Now that you have your grayscale ramps, you can set up the image. The following code opens the image using `Pillow` and splits it into a grid:

```
# open the image and convert to grayscale

image = Image.open(fileName).convert("L")

# store the image dimensions

❶ W, H = image.size[0], image.size[1]

# compute the tile width

❷ w = W/cols

# compute the tile height based on the aspect ratio and scale of the
font

❸ h = w/scale

# compute the number of rows to use in the final grid

❹ rows = int(H/h)
```

First, `Image.open()` opens the input image file, and `Image.convert()` converts the image to grayscale. The `"L"` stands for *lumi-*

nance, a measure of the brightness of an image. You store the width and height (measured in pixels) of the input image ❶. Then you compute the width of a tile for the number of columns (`cols`) specified by the user ❷. (The program uses a default of 80 columns if the user doesn't set another value in the command line.) You use floating-point, not integer, division to avoid truncation errors while calculating the dimensions of the tiles.

Once you know the width of a tile, you compute its height using the vertical scale factor passed in as `scale` ❸. This way, each tile will match the aspect ratio of the font you're using to display the text so that the final image won't be distorted. The value for `scale` can be passed in as an argument, or it's set to a default of `0.43`, which works well for displaying the result in Courier. Having calculated the height of each row, you compute the number of rows in the grid ❹.

Computing the Average Brightness

Next, you need a way to compute the average brightness for a tile in the grayscale image. The function `getAverageL()` does the job.

```
def getAverageL(image):

    # get the image as a numpy array

  ❶ im = np.array(image)

    # get the dimensions

    w,h = im.shape

    # get the average

  ❷ return np.average(im.reshape(w*h))
```

The image tile is passed into the function as a PIL `Image` object. You convert the image into a `numpy` array ❶, at which point `im` becomes a two-dimensional array containing the brightness values of the image's pixels. You store the dimensions (width and height) of the array and then use `numpy.reshape()` to convert the two-dimensional array into a flat one-dimensional array whose length is a product of the original array's width and height (`w*h`). You pass the reshaped array to `numpy.average()`, which sums the array values and computes the average brightness level of the entire image tile ❷.

Generating the ASCII Content from the Image

The main part of the program generates the ASCII content from the image:

```
# an ASCII image is a list of character strings

❶ aimg = []

# generate the list of tile dimensions

❷ for j in range(rows):

    y1 = int(j*h)

    y2 = int((j+1)*h)

    # correct the last tile

    if j == rows-1:

      ❸ y2 = H

    # append an empty string

  ❹ aimg.append("")
```

```
❺ for i in range(cols):
    # crop the image to fit the tile
    x1 = int(i*w)
    x2 = int((i+1)*w)
    # correct the last tile
    if i == cols-1:
        x2 = W
    # crop the image to extract the tile into another Image object
❻ img = image.crop((x1, y1, x2, y2))
    # get the average luminance
❼ avg = int(getAverageL(img))
    # look up the ASCII character for grayscale value (avg)
    if moreLevels:
      ❽ gsval = gscale1[int((avg*69)/255)]
    else:
      ❾ gsval = gscale2[int((avg*9)/255)]
    # append the ASCII character to the string
❿ aimg[j] += gsval
```

In this section of the program, the ASCII image is first stored as a list of strings, which you initialize ❶. Next, you iterate through the rows of image tiles ❷, calculating the top and bottom y-coordinates of each image tile in a given row as *y1* and *y2*. These are floating-point calculations, but you truncate them to integers before passing them to an image-cropping method.

Next, because dividing the image into tiles creates edge tiles of the same size only when the image width is an integer multiple of the number of columns, you correct for the bottom y-coordinate of the tiles in the last row by setting the y-coordinate to the image's actual height (`H`) ❸. By doing so, you ensure that the bottom edge of the image isn't truncated.

You add an empty string into the ASCII image list as a compact way to represent the current image row ❹. You'll fill in this string next. Essentially, you're treating the string as a list of characters that you can append to. Then you iterate over all the tiles in a given row of the image, column by column ❺. You compute the left and right x-coordinates of each tile as *x1* and *x2*. When you get to the last tile in the row, you set the right x-coordinate to the width of the image (`W`), for the same reasons you corrected the final y-coordinate to the image's height.

You've now calculated (*x1*, *y1*) and (*x2*, *y2*), the coordinates of the top-left and bottom-right corners of the current image tile. You pass these coordinates to `image.crop()` to extract the tile from the complete image ❻. Then you pass that tile (which takes the form of a PIL `Image` object) to the `getAverageL()` function ❼, defined in **"Computing the Average Brightness" on page 105**, to get the average brightness of the tile. You scale the average brightness value from [0, 255] to [0, 9], the range of values for the default 10-level grayscale ramp ❾. You then use `gscale2` (the stored ramp string) as a lookup table for the relevant ASCII character. The line at ❽ is similar, except it scales the brightness value to the [0, 69] range of the 70-level grayscale ramp. This line will

be used only when the `moreLevels` command line flag has been set. Finally, you append the looked-up ASCII character, `gsval`, to the text row ❿, and the code loops until all rows are processed.

Creating Command Line Options

Next, define some command line options for the program. This code uses the built-in `argparse.ArgumentParser` class:

```
parser = argparse.ArgumentParser(description="descStr")

# add expected arguments

parser.add_argument('--file', dest='imgFile', required=True)

parser.add_argument('--scale', dest='scale', required=False)

parser.add_argument('--out', dest='outFile', required=False)

parser.add_argument('--cols', dest='cols', required=False)

parser.add_argument('--morelevels', dest='moreLevels',
action='store_true')
```

You include the following options:

--file Specifies the image file to input. This is the only required argument.

--scale Sets the vertical scale factor for a font other than Courier.

--out Sets the output filename for the generated ASCII art. Defaults to *out.txt*.

--cols Sets the number of text columns in the ASCII output.

--morelevels Selects the 70-level grayscale ramp instead of the default 10-level ramp.

Writing the ASCII Art Strings to a Text File

Finally, take the generated list of ASCII character strings and write those strings to a text file:

```
# open a new text file

❶ f = open(outFile, 'w')

# write each string in the list to the new file

❷ for row in aimg:

    f.write(row + '\n')

# clean up

❸ f.close()
```

You use the built-in `open()` function to open a new text file for writing ❶. Then you iterate through each string in the `aimg` list and write it to the file ❷. When you're done, you close the file object to release system resources ❸.

Running the ASCII Art Generator

To run your finished program, enter a command like the following one, replacing `data/robot.jpg` with the relative path to the image file you want to use:

```
$ python ascii.py --file data/robot.jpg --cols 100
```

Figure 6-3 shows the ASCII art that results from sending the image *robot.jpg* (at the left). Try adding the `--morelevels` option to see how the 70-level grayscale ramp compares to the 10-level ramp.

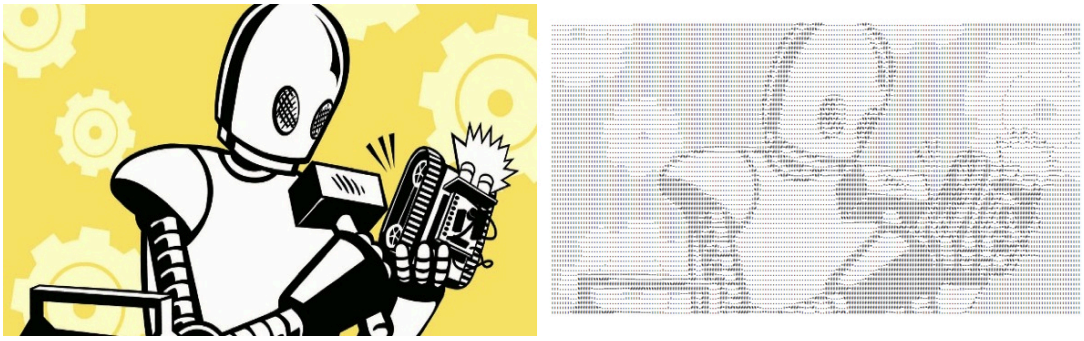

***Figure 6-3:** A sample run of* ascii.py

Now you're all set to create your own ASCII art!

Summary

In this project, you learned how to generate ASCII art from any input image. In the process, you learned how to split an image into a grid of tiles, how to compute the average brightness value of each tile, and how to replace each tile with a character based on the brightness values. Have fun creating your own ASCII art!

Experiments!

Here are some ideas for exploring ASCII art further:

1. Run the program with the command line option `--scale 1.0`. How does the resulting image look? Experiment with different values for `scale`. Copy the output to a text editor and try setting the text to different fixed-width fonts to see how doing so affects the appearance of the final image.
2. Add a command line option `--invert` to the program to invert the generated ASCII images so that black appears white, and vice versa.

(Hint: try subtracting the tile brightness value from 255 during lookup.)

3. 3. In this project, you created lookup tables for grayscale values based on two hardcoded character ramps. Implement a command line option to pass in a different character ramp to create the ASCII art, like so:

```
$ python ascii.py --map "@$%^`."
```

This should create the ASCII output using the given six-character ramp, where `@` maps to a brightness value of 0 and `.` maps to a value of 255.

The Complete Code

Here is the complete ASCII art program.

```
"""

ascii.py

A Python program that convert images to ASCII art.

Author: Mahesh Venkitachalam

"""

import sys, random, argparse

import numpy as np

import math

from PIL import Image

# grayscale level values from:

# http://paulbourke.net/dataformats/asciiart/
```

```
    # 70 levels of gray

    gscale1 = "$@B%8&WM#*oahkbdpqwmZO0QLCJUYXzcvunxrjft/\|()1{}
    []?-_+~<>i!lI;:,\"^`'. "

    # 10 levels of gray

    gscale2 = '@%#*+=-:. '

    def getAverageL(image):

        """

        given PIL Image, return average value of grayscale value

        """

        # get image as numpy array

        im = np.array(image)

        # get shape

        w,h = im.shape

        # get average

        return np.average(im.reshape(w*h))

    def convertImageToAscii(fileName, cols, scale, moreLevels):

        """

        given Image and dims (rows, cols) returns an m*n list of Images

        """
```

```
# declare globals

global gscale1, gscale2

# open image and convert to grayscale

image = Image.open(fileName).convert('L')

# store dimensions

W, H = image.size[0], image.size[1]

print("input image dims: {} x {}".format(W, H))

# compute width of tile

w = W/cols

# compute tile height based on aspect ratio and scale

h = w/scale

# compute number of rows

rows = int(H/h)

print("cols: {}, rows: {}".format(cols, rows))

print("tile dims: {} x {}".format(w, h))

# check if image size is too small

if cols > W or rows > H:

    print("Image too small for specified cols!")

    exit(0)
```

```
# an ASCII image is a list of character strings
aimg = []
# generate list of dimensions
for j in range(rows):
    y1 = int(j*h)
    y2 = int((j+1)*h)
    # correct last tile
    if j == rows-1:
        y2 = H
    # append an empty string
    aimg.append("")
    for i in range(cols):
        # crop image to tile
        x1 = int(i*w)
        x2 = int((i+1)*w)
        # correct last tile
        if i == cols-1:
            x2 = W
        # crop image to extract tile
```

```
            img = image.crop((x1, y1, x2, y2))
            # get average luminance
            avg = int(getAverageL(img))
            # look up ASCII char
            if moreLevels:
                gsval = gscale1[int((avg*69)/255)]
            else:
                gsval = gscale2[int((avg*9)/255)]
            # append ASCII char to string
            aimg[j] += gsval
    # return image
    return aimg
# main() function
def main():
    # create parser
    descStr = "This program converts an image into ASCII art."
    parser = argparse.ArgumentParser(description=descStr)
    # add expected arguments
    parser.add_argument('--file', dest='imgFile', required=True)
```

```
    parser.add_argument('--scale', dest='scale', required=False)
    parser.add_argument('--out', dest='outFile', required=False)
    parser.add_argument('--cols', dest='cols', required=False)
    parser.add_argument('--
morelevels',dest='moreLevels',action='store_true')
    # parse args
    args = parser.parse_args()
    imgFile = args.imgFile
    # set output file
    outFile = 'out.txt'
    if args.outFile:
        outFile = args.outFile
    # set scale default as 0.43, which suits a Courier font
    scale = 0.43
    if args.scale:
        scale = float(args.scale)
    # set cols
    cols = 80
    if args.cols:
```

```
        cols = int(args.cols)

    print('generating ASCII art...')

    # convert image to ASCII text

    aimg = convertImageToAscii(imgFile, cols, scale, args.moreLevels)

    # open file

    f = open(outFile, 'w')

    # write to file

    for row in aimg:

        f.write(row + '\n')

    # clean up

    f.close()

    print("ASCII art written to {}.".format(outFile))

# call main

if __name__ == '__main__':

    main()
```
